

Scenario

In recent days, ShopSphere, a prominent online retail platform, has experienced unusual administrative login activity during late-night hours. These logins coincide with an influx of customer complaints about unexplained account anomalies, raising concerns about a potential security breach. Initial observations suggest unauthorized access to administrative accounts, potentially indicating deeper system compromise.

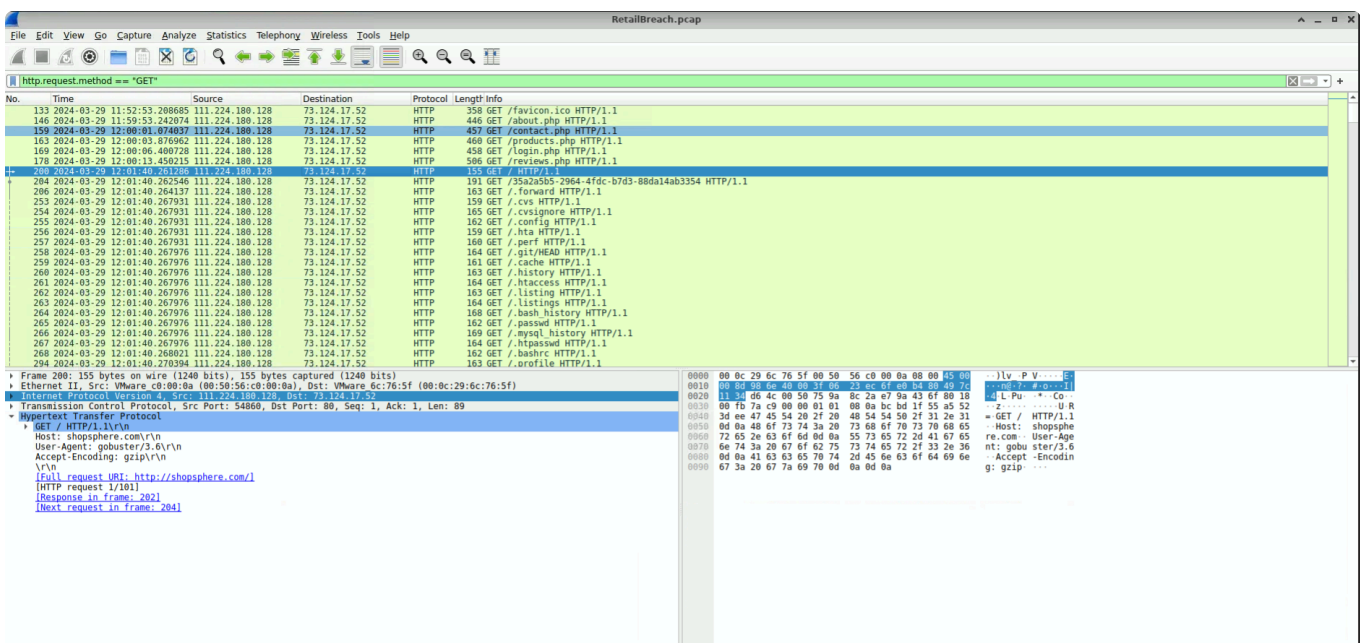
Your mission is to investigate the captured network traffic to determine the nature and source of the breach. Identifying how the attackers infiltrated the system and pinpointing their methods will be critical to understanding the attack's scope and mitigating its impact.

Task # 01

Identifying an attacker's IP address is crucial for mapping the attack's extent and planning an effective response. What is the attacker's IP address?

As it was mentioned that directory brute-force technique has been used, so I filtered all the GET requests. Analyzing requests showed that most of the requests were from single IP which was attacker.

```
http.request.method == "GET"
```



Task # 02

The attacker used a directory brute-forcing tool to discover hidden paths. Which tool did the attacker use to perform the brute-forcing?

I viewed the details of a GET request coming from the attacker's IP. User-Agent of the request showed that `gobuster` was used as brute-forcing tool.

Task # 03

Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages viewed by users. Can you specify the XSS payload that the attacker used to compromise the integrity of the web application?

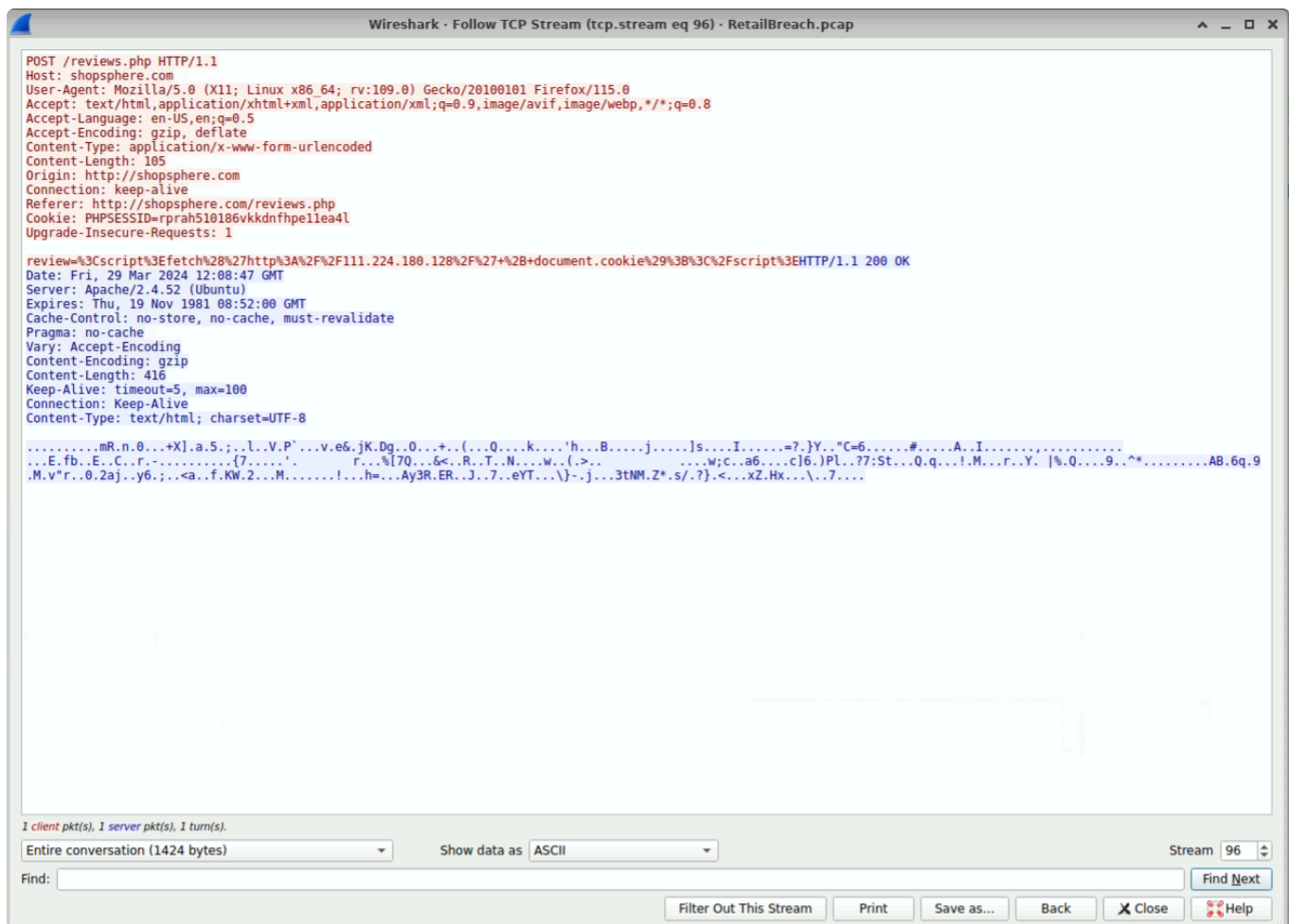
Common XSS attacks use `<script>` method, so I filtered http traffic coming from attacker's IP and which contains `script` in the request. There was only one POST request among the filtered traffic. I clicked on the request and followed tcp stream which showed the encoded url.

http contains "script" && ip.src == 111.224.180.128

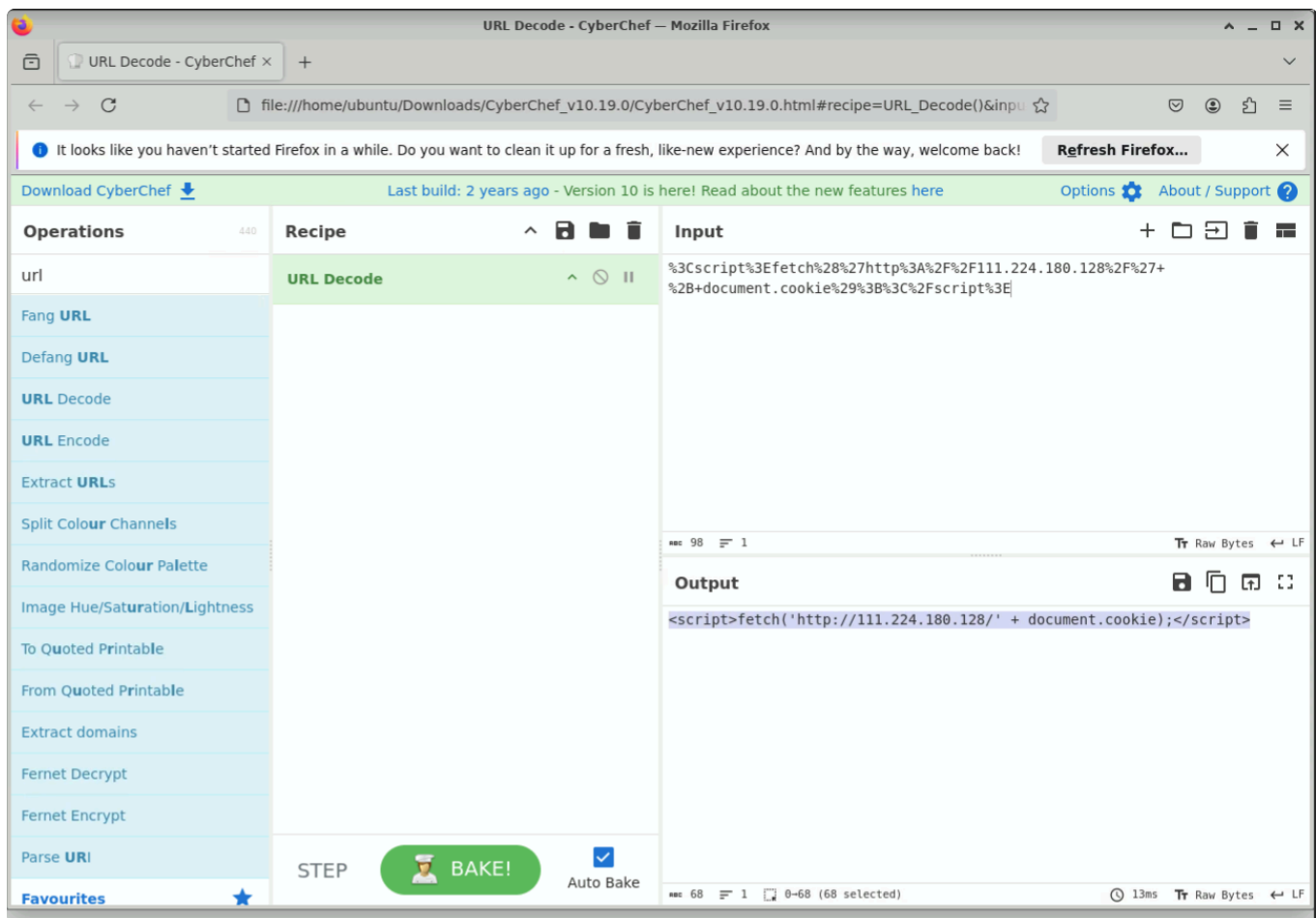
The screenshot shows the Wireshark interface with the packet list filtered to show only HTTP requests containing "script" and originating from IP 111.224.180.128. The selected packet is a POST request to `reviews.php`. The details pane shows the following information:

- Frame 10058: 712 bytes on wire (5696 bits), 712 bytes captured (5696 bits) on Ethernet II, Src: VMware c8:00:0a:00:50:56, Dst: VMware 6c:78:5f:00:0c:29:6c:76:5f
- Internet Protocol Version 4 (IP): 111.224.180.128 -> 192.168.1.100
- Transmission Control Protocol, Src Port: 51380, Dst Port: 80, Seq: 1, Ack: 1, Len: 646
- Hypertext Transfer Protocol
 - POST /reviews.php HTTP/1.1
 - Host: shopspere.com
 - User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20180813 Firefox/115.0
 - Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
 - Accept-Language: en-US,en;q=0.5
 - Accept-Encoding: gzip, deflate
 - Content-Type: application/x-www-form-urlencoded
 - Content-Length: 105
 - Origin: http://shopspere.com
 - Connection: keep-alive
 - Referer: http://shopspere.com/reviews.php
 - Cookie: PHPSESSID=prah51818vknfndfhpellea4
 - Upgrade-Insecure-Requests: 1
 - Full request URI: http://shopspere.com/reviews.php
 - HTTP request 1/1
 - Response in frame: 10060
 - File Data: 105 bytes
 - HTML Form URL Encoded: application/x-www-form-urlencoded

The packet bytes pane shows the raw data of the request, including the POST body which is URL-encoded.



I copied the encoded url and decoded it using Cyberchef which resulted in payload used by the attacker.

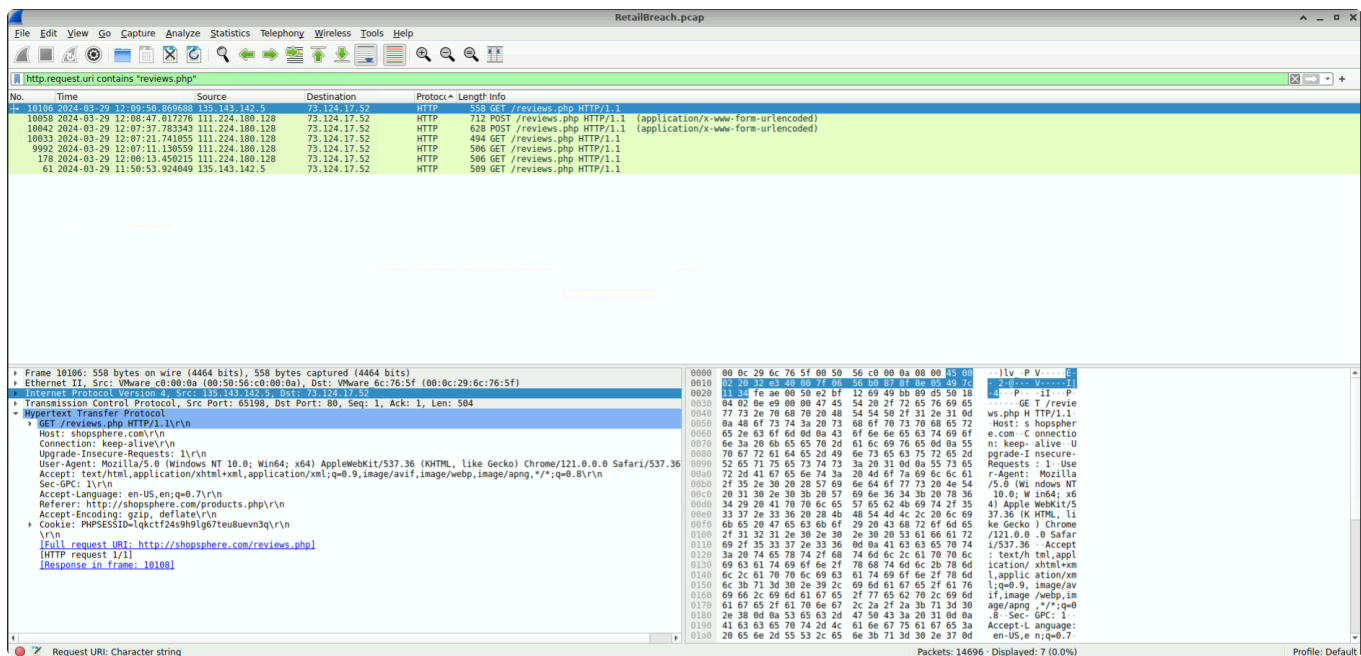


Task # 04

Pinpointing the exact moment an admin user encounters the injected malicious script is crucial for understanding the timeline of a security breach. Can you provide the UTC timestamp when the admin user first visited the page containing the injected malicious script?

Since XSS payload showed that the attacker posted it in review, so i filtered http requests containg reviews.php page. As there are two GET requests coming from admin's IP, attacker uploaded payload on 12:08 which confirmed that the request after this time would be the one affecting the admin user. So request on 12:09 was the answer.

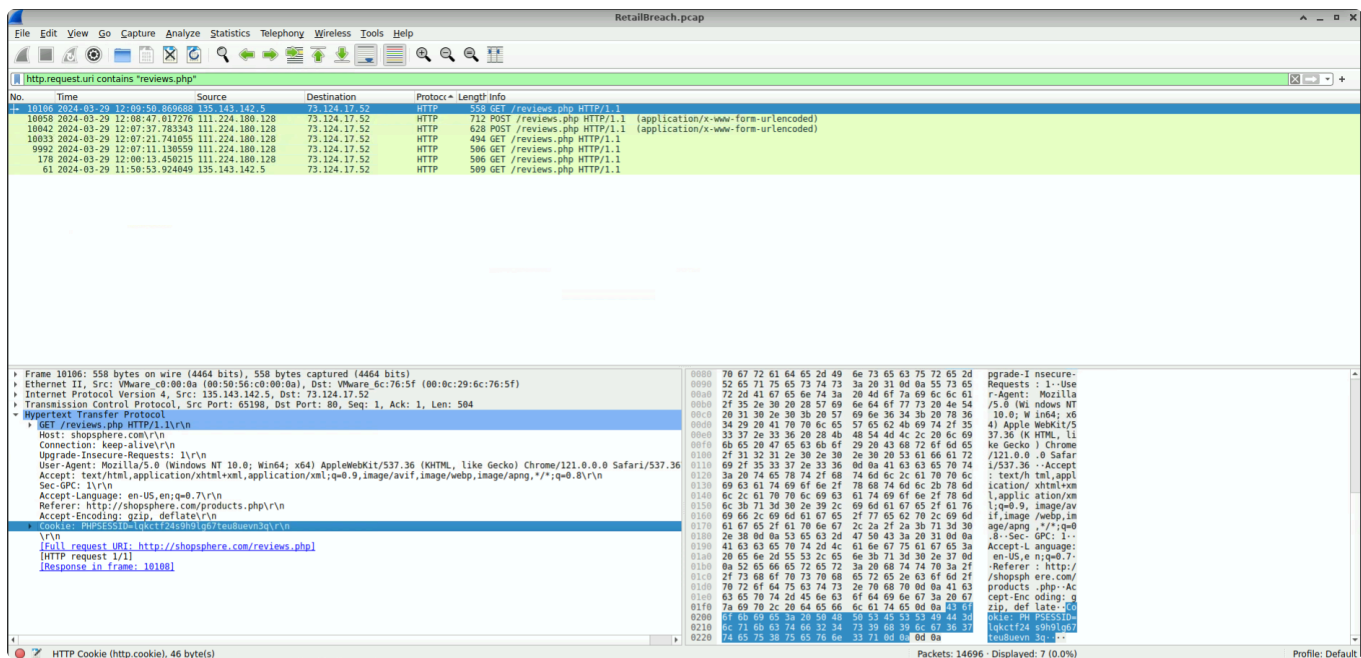
```
http.request.uri contains "reviews.php"
```



Task # 05

The theft of a session token through XSS is a serious security breach that allows unauthorized access. Can you provide the session token that the attacker acquired and used for this unauthorized access?

In the same GET request identified in previous task, session token can be found by viewing the details.



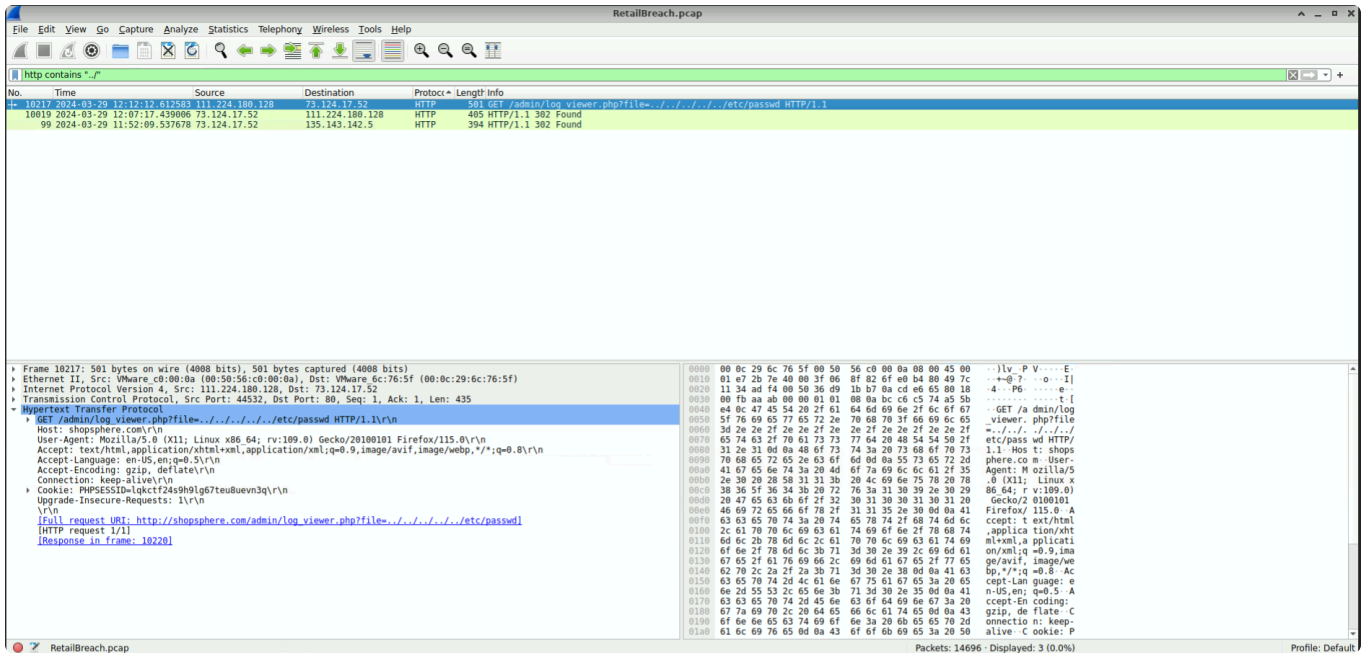
Task # 06

Identifying which scripts have been exploited is crucial for mitigating vulnerabilities in a web application. What is the name of the script that

was exploited by the attacker?

For this task, I used the hint which stated that the file outside of normal access was used. Hence, I filtered traffic consisting of directory traversal.

http contains "../"



Task # 07

Exploiting vulnerabilities to access sensitive system files is a common tactic used by attackers. Can you identify the specific payload the attacker used to access a sensitive system file?

The request identified in the last task contained the payload which accessed /etc/passwd file using directory traversal.